分子可视化工具 / Molecular Visualization Tool

一个功能完整的、双语（中英文）的分子可视化Web应用，支持多种分子数据格式（.log, .mol, .cif），提供交互式3D展示分子结构和分子构建器功能。

A full-featured bilingual (English/Chinese) molecular visualization web application that supports multiple molecular data formats (.log, .mol, .cif), providing interactive 3D molecular structure display and molecule builder functionality.

📋 目录 / Table of Contents

- 主要功能
- 技术架构
- 详细安装指南
- 项目结构详解
- 前后端实现细节
- 使用说明
- 算法原理
- 故障排除

✨ 主要功能 / Key Features

📊 多格式数据解析与可视化

- **支持三种分子数据格式**：
  - `.log` - Gaussian 计算化学日志文件（包含完整的电子密度信息）
  - `.mol` - MDL MOL 文件格式（包含原子坐标和键连接）
  - `.cif` - 晶体学信息文件格式（包含晶体结构数据）
- **智能化学键识别**：自动识别单键、双键、三键
- **电子云可视化**：基于量子化学计算结果生成高低密度电子云等值面
- **CPK 配色方案**：使用标准 CPK 着色方案渲染原子

🎮 交互式 3D 控制

- 左键拖拽：旋转分子视图
- 滚轮缩放：调整视图大小
- 右键拖拽：平移视图
- 按 S 键：保存当前视图截图

🔨 分子构建器

- 交互式添加原子（C, H, O, N）
- 创建和删除化学键
- 实时化合价验证
- 错误原子高亮显示

🌍 完整双语支持

- 中英文界面一键切换
- 所有提示信息双语显示

🏗️ 技术架构 / Technical Architecture

整体架构

```
┌──────────────────────────────────────────────────────────────┐
│                    用户浏览器 / Browser                       │
│  ┌──────────────────────────────────────────────────────┐    │
│  │          前端 / Frontend (JavaScript)                 │    │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐ │    │
│  │  │ index.html   │  │    ui.js     │  │builder.js    │ │    │
│  │  └──────────────┘  └──────────────┘  └──────────────┘ │    │
│  │  ┌──────────────┐  ┌──────────────────────────────┐  │    │
│  │  │visualizer.js │  │    Three.js (3D 渲染)        │  │    │
│  │  └──────────────┘  └──────────────────────────────┘  │    │
│  └──────────────────────────────────────────────────────┘    │
│                         ↕ HTTP/JSON                          │
└──────────────────────────────────────────────────────────────┘
                              ↕
┌──────────────────────────────────────────────────────────────┐
│              后端服务器 / Backend (Python)                    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │               Flask Web 框架                          │    │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐ │    │
│  │  │   app.py     │  │ parser.py    │  │calculator.py │ │    │
│  │  │ (路由控制)   │  │ (文件解析)   │  │ (计算模块)   │ │    │
│  │  └──────────────┘  └──────────────┘  └──────────────┘ │    │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │     科学计算库 (NumPy, SciPy, scikit-image)           │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
                              ↕
┌──────────────────────────────────────────────────────────────┐
│            数据文件 / Data Files                              │
│     .log (Gaussian)  .mol (MDL)  .cif (Crystallography)      │
└──────────────────────────────────────────────────────────────┘
```

技术栈详情

后端 / Backend

- **Python 3.8+**: 核心编程语言
- **Flask 2.3.0**: 轻量级 Web 框架
- **NumPy 1.24.3**: 数值计算和矩阵操作
- **SciPy 1.10.1**: 科学计算库
- **scikit-image 0.21.0**: Marching Cubes 等值面生成
- **gemmi ≥0.6.0** (可选): 高级 CIF 文件解析
- **flask-cors 4.0.0**: 跨域资源共享支持

前端 / Frontend

- **HTML5/CSS3**: 现代化界面结构和样式
- **JavaScript ES6+**: 交互逻辑实现
- **Three.js r128**: 3D 图形渲染引擎
- **WebGL**: GPU 加速的 3D 渲染
- **OrbitControls**: 相机控制插件

🚀 详细安装指南 / Detailed Installation Guide

系统要求

组件	最低要求	推荐配置
操作系统	Windows 7/macOS 10.12/Linux	Windows 10/11, macOS 12+, Ubuntu 20.04+
Python	3.8	3.9 或 3.10
内存	4 GB RAM	8 GB RAM 或更多
浏览器	Chrome 90+, Firefox 88+, Edge 90+	最新版本
GPU	支持 WebGL 1.0	支持 WebGL 2.0

步骤 1：获取项目代码

方式 A：使用 Git 克隆（推荐）

```
git clone <repository-url>
cd fzrj
```

方式 B：直接下载

1. 下载项目压缩包
2. 解压到本地目录
3. 进入项目根目录

步骤 2：配置 Python 环境

2.1 检查 Python 版本

```
python --version
# 或
python3 --version
```

确保版本 ≥ 3.8

2.2 创建虚拟环境（推荐）

Windows:

```
python -m venv venv
venv\Scripts\activate
```

macOS/Linux:

```
python3 -m venv venv
source venv/bin/activate
```

虚拟环境激活后，命令提示符前会显示 `(venv)`

2.3 升级 pip

```
python -m pip install --upgrade pip
```

步骤 3：安装 Python 依赖

```
pip install -r requirements.txt
```

依赖包说明：

- `Flask==2.3.0` - Web 框架
- `flask-cors==4.0.0` - 跨域支持
- `numpy==1.24.3` - 数值计算
- `scipy==1.10.1` - 科学计算
- `scikit-image==0.21.0` - 图像处理（等值面生成）
- `gemmi>=0.6.0` - CIF 文件解析（可选，建议安装）

常见安装问题：

1. **网络问题**：使用国内镜像源

```
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

2. **编译错误**：某些包需要 C++ 编译器
   - Windows: 安装 Visual Studio Build Tools
   - macOS: 安装 Xcode Command Line Tools: `xcode-select --install`
   - Linux: 安装 gcc: `sudo apt-get install build-essential`

步骤 4：验证安装

运行测试脚本：

```
python test_installation.py
```

如果输出 ✅ 所有依赖安装成功！ ，则安装完成。

步骤 5：启动应用

5.1 启动后端服务器

方式 A：从 backend 目录启动

```
cd backend
python app.py
```

方式 B：从项目根目录启动

```
python backend/app.py
```

成功启动后，会显示：

```
============================================================
分子可视化工具 - Flask后端服务器
============================================================
服务器运行在: http://localhost:5000
请在浏览器中打开上述地址访问应用
============================================================
```

5.2 访问应用

打开浏览器，访问：

```
http://localhost:5000
```

或者

```
http://127.0.0.1:5000
```

步骤 6：验证功能

1. **测试文件上传**：
   - 点击 "选择分子文件" 按钮
   - 选择 `data/NH3.log` （示例氨气分子）
   - 查看 3D 渲染结果
2. **测试语言切换**：
   - 点击 "Switch to English" 按钮
   - 界面应切换为英文
3. **测试构建模式**：
   - 点击 "启用构建模式"
   - 选择原子类型并在 3D 视图中点击添加

📁 项目结构详解 / Project Structure Details

```
fzrj/                                   # 项目根目录
│
├── backend/                            # Python 后端目录
│   ├── __init__.py                     # Python 包初始化文件
│   ├── __pycache__/                    # Python 字节码缓存（自动生成）
│   │
│   ├── app.py                          # Flask 主应用文件 [核心]
│   │   ├── 功能：定义 API 路由
│   │   ├── 路由 1: GET  /              → 返回前端页面
│   │   ├── 路由 2: POST /api/parse     → 解析上传的分子文件
│   │   ├── 路由 3: GET  /api/test_files → 列出测试文件
│   │   └── 路由 4: GET  /api/load_test/<filename> → 加载测试文件
│   │
│   ├── parser.py                       # 分子文件解析模块 [核心]
│   │   ├── parse_gaussian_log()        → 解析 .log 文件
│   │   ├── parse_mol_file()            → 解析 .mol 文件
│   │   ├── parse_cif_file()            → 解析 .cif 文件
│   │   ├── parse_cif_simple()          → 简化 CIF 解析器
│   │   └── parse_molecule_file()       → 统一解析接口
│   │
│   ├── calculator.py                   # 计算模块 [核心]
│   │   ├── calculate_bonds()           → 计算化学键
│   │   ├── determine_bond_type()       → 判断键类型
│   │   └── generate_isosurface()       → 生成电子云等值面
│   │
│   └── uploads/                        # 上传文件临时存储目录
│       └── *.log, *.mol, *.cif         # 用户上传的文件
│
├── frontend/                           # 前端静态文件目录
│   ├── index.html                      # 主 HTML 页面 [入口]
│   │   ├── 结构：左侧控制面板 + 右侧 3D 画布
│   │   ├── 包含：文件上传、显示控制、构建器UI
│   │   └── 引入：Three.js, visualizer.js, ui.js, builder.js
│   │
│   ├── visualizer.js                   # 3D 可视化模块 [核心]
│   │   ├── initScene()                 → 初始化 Three.js 场景
│   │   ├── renderMolecule()            → 渲染分子
│   │   ├── renderAtoms()               → 渲染原子球体
```

```
│   │   ├── renderBonds()              → 渲染化学键圆柱
│   │   ├── renderIsosurfaces()        → 渲染电子云等值面
│   │   ├── toggleIsosurface()         → 切换等值面显示
│   │   ├── saveScreenshot()           → 保存截图
│   │   └── fitCameraToMolecule()      → 自动调整相机
│   │
│   ├── ui.js                           # UI 交互和多语言模块 [核心]
│   │   ├── languages                  → 中英文文本配置
│   │   ├── setLanguage()              → 切换语言
│   │   ├── getText()                  → 获取多语言文本
│   │   ├── showInfo()                 → 显示信息提示
│   │   ├── loadFile()                 → 上传并解析文件
│   │   └── 事件监听器                 → 按钮点击、文件选择等
│   │
│   └── builder.js                      # 分子构建器模块 [核心]
│       ├── toggleBuilderMode()        → 启用/关闭构建模式
│       ├── addAtom()                  → 添加原子
│       ├── createBond()               → 创建化学键
│       ├── deleteAtom()               → 删除原子
│       ├── validateMolecule()         → 验证化合价
│       └── renderBuiltMolecule()      → 渲染构建的分子
│
├── data/                               # 测试数据文件目录
│   ├── *.log                           # Gaussian 日志文件 (100+ 个)
│   │   ├── NH3.log                    → 氨气分子
│   │   ├── AP.log                     → 大分子示例
│   │   └── ...                        → 其他分子
│   │
│   ├── 71358.mol                       # MDL MOL 格式示例
│   └── 1e7u.cif                        # CIF 格式示例 (蛋白质结构)
│
├── requirements.txt                    # Python 依赖包列表
│   ├── Flask==2.3.0
│   ├── flask-cors==4.0.0
│   ├── numpy==1.24.3
│   ├── scipy==1.10.1
│   ├── scikit-image==0.21.0
│   └── gemmi>=0.6.0
│
├── test_installation.py                # 安装验证脚本
├── test_multi_format.py                # 多格式解析测试脚本
├── test_complete_flow.py               # 完整流程测试脚本
│
└── README.md                           # 项目文档 (本文件)
```

文件大小说明

目录/文件类型	数量	总大小
Python 代码 (.py)	~10 个	~50 KB
JavaScript 代码 (.js)	3 个	~40 KB
HTML/CSS	1 个	~15 KB
测试数据 (.log)	100+ 个	~50 MB

目录/文件类型	数量	总大小
测试数据 (.mol, .cif)	2 个	~190 MB

💻 前后端实现细节 / Frontend & Backend Implementation

后端实现 (Python + Flask)

1. Flask 应用架构 (`backend/app.py`)

```
# 核心流程
Flask 应用启动
    ↓
定义 API 路由
    ↓
监听 HTTP 请求
    ↓
调用解析器 (parser.py)
    ↓
调用计算器 (calculator.py)
    ↓
返回 JSON 数据给前端
```

关键 API 端点:

端点	方法	功能	请求	响应
`/`	GET	返回主页	-	HTML 页面
`/api/parse`	POST	解析上传文件	FormData(file)	JSON{atoms, bonds, isosurfaces}
`/api/test_files`	GET	列出测试文件	-	JSON{files: [...]}
`/api/load_test/<filename>`	GET	加载测试文件	filename	JSON{atoms, bonds, isosurfaces}

工作流程示例:

```
# 1. 用户上传文件
POST /api/parse
    ↓
file = request.files['file']
    ↓
# 2. 保存文件
filepath = 'uploads/' + filename
file.save(filepath)
    ↓
```

```
# 3. 解析文件
parsed_data = parse_molecule_file(filepath)
# 返回: {atoms: [...], density_matrix: [...]}
    ↓
# 4. 计算化学键
bonds = calculate_bonds(atoms)
    ↓
# 5. 生成电子云
isosurfaces = generate_isosurface(atoms, density_matrix)
    ↓
# 6. 返回 JSON
return jsonify({
    'atoms': atoms,
    'bonds': bonds,
    'isosurfaces': isosurfaces
})
```

2. 文件解析器 (`backend/parser.py`)

支持三种格式的解析:

A. Gaussian .log 文件解析

```
def parse_gaussian_log(file_path):
    # 步骤 1: 读取文件内容
    content = read_file(file_path)

    # 步骤 2: 提取原子坐标
    # 正则表达式匹配 "Standard orientation" 块
    pattern = r'Standard orientation:.*?-{5,}(.*?)-{5,}'
    matches = re.finditer(pattern, content)
    last_match = matches[-1]  # 取最后一个 (优化后的结构)

    # 解析每一行: Center Number, Atomic Number, X, Y, Z
    for line in last_match:
        atomic_num = int(parts[1])
        x, y, z = float(parts[3]), float(parts[4]), float(parts[5])
        element = ATOMIC_NUMBERS[atomic_num]
        atoms.append({'element': element, 'position': [x, y, z]})

    # 步骤 3: 提取电子密度矩阵
    # 查找 "Population analysis using the SCF density"
    # 提取 "Condensed to atoms (all electrons):" 后的矩阵
    density_matrix = parse_density_matrix(content)

    return {'atoms': atoms, 'density_matrix': density_matrix}
```

B. MDL .mol 文件解析

```
def parse_mol_file(file_path):
    lines = read_lines(file_path)

    # 第 4 行包含原子数和键数
    counts_line = lines[3]
    n_atoms = int(counts_line.split()[0])
    n_bonds = int(counts_line.split()[1])
```

```
    # 从第 5 行开始解析原子块
    # 格式: X Y Z Element ...
    for i in range(4, 4 + n_atoms):
        parts = lines[i].split()
        x, y, z = float(parts[0]), float(parts[1]), float(parts[2])
        element = parts[3]
        atoms.append({'element': element, 'position': [x, y, z]})

    # .mol 文件无电子密度，使用简化矩阵
    density_matrix = np.eye(n_atoms) * 2.0

    return {'atoms': atoms, 'density_matrix': density_matrix}
```

C. CIF 文件解析

```
def parse_cif_file(file_path):
    try:
        # 优先使用 gemmi 库解析
        import gemmi
        doc = gemmi.cif.read_file(file_path)
        block = doc.sole_block()

        # 查找 _atom_site loop
        loop = block.find_loop('_atom_site_label')
        for row in loop:
            element = row[type_symbol_index]
            x = float(row[Cartn_x_index])
            y = float(row[Cartn_y_index])
            z = float(row[Cartn_z_index])
            atoms.append({'element': element, 'position': [x, y, z]})

    except ImportError:
        # 降级使用简化解析器
        atoms = parse_cif_simple(content)

    return {'atoms': atoms, 'density_matrix': ...}
```

统一接口:

```
def parse_molecule_file(file_path):
    ext = os.path.splitext(file_path)[1].lower()

    if ext == '.log':
        return parse_gaussian_log(file_path)
    elif ext == '.mol':
        return parse_mol_file(file_path)
    elif ext == '.cif':
        return parse_cif_file(file_path)
    else:
        raise ValueError(f"不支持的格式: {ext}")
```

3. 计算模块 (`backend/calculator.py`)

A. 化学键计算

```python
def calculate_bonds(atoms):
    bonds = []

    # 遍历所有原子对
    for i in range(len(atoms)):
        for j in range(i + 1, len(atoms)):
            # 计算距离
            pos1 = np.array(atoms[i]['position'])
            pos2 = np.array(atoms[j]['position'])
            distance = np.linalg.norm(pos1 - pos2)

            # 判断键类型
            bond_type = determine_bond_type(
                atoms[i]['element'],
                atoms[j]['element'],
                distance
            )

            if bond_type:
                bonds.append({
                    'atom1_idx': i,
                    'atom2_idx': j,
                    'type': bond_type
                })

    return bonds
```

键长判断表：

原子对	单键 (Å)	双键 (Å)	三键 (Å)
C-C	1.45-1.55	1.30-1.40	1.15-1.25
C-N	1.40-1.50	1.25-1.35	1.10-1.20
C-O	1.38-1.48	1.18-1.28	-
C-H	1.05-1.15	-	-
N-H	0.98-1.08	-	-
O-H	0.92-1.02	-	-

B. 电子云等值面生成（改进算法）

```python
def generate_isosurface(atoms, density_matrix, grid_size=50):
    # 步骤 1：创建 3D 网格
    min_coords = atoms_positions.min(axis=0) - 2.0
    max_coords = atoms_positions.max(axis=0) + 2.0
    X, Y, Z = np.meshgrid(
        np.linspace(min_coords[0], max_coords[0], grid_size),
        np.linspace(min_coords[1], max_coords[1], grid_size),
        np.linspace(min_coords[2], max_coords[2], grid_size)
    )

    # 步骤 2：标准化权重（改进点）
```

```
    diag_values = [density_matrix[i][i] for i in range(len(atoms))]
    max_diag = max(diag_values)
    min_diag = min(diag_values)

    # 步骤 3: 计算每个网格点的电子密度
    density_field = np.zeros((grid_size, grid_size, grid_size))
    alpha = 1.0  # 衰减系数 (减小以扩大电子云范围)

    for i, atom in enumerate(atoms):
        # 归一化权重到 0.5-1.5, 避免大原子主导
        normalized_weight = (diag_values[i] - min_diag) / (max_diag - min_diag)
        weight = 0.5 + normalized_weight

        # 计算距离
        dx = X - atom['position'][0]
        dy = Y - atom['position'][1]
        dz = Z - atom['position'][2]
        r_squared = dx**2 + dy**2 + dz**2

        # 高斯型贡献
        density_field += weight * np.exp(-alpha * r_squared)

    # 步骤 4: 对数归一化 (改进点: 压缩动态范围)
    density_field = np.log1p(density_field)
    density_field = density_field / density_field.max()

    # 步骤 5: Marching Cubes 算法生成等值面
    # 低密度 (阈值 0.02) - 外层电子云
    verts_low, faces_low = marching_cubes(density_field, level=0.02)

    # 高密度 (阈值 0.5) - 内层电子云
    verts_high, faces_high = marching_cubes(density_field, level=0.5)

    return {
        'low_density': {'verts': verts_low, 'faces': faces_low},
        'high_density': {'verts': verts_high, 'faces': faces_high}
    }
```

算法改进说明:

改进点	原始算法	改进算法	效果
衰减系数	α = 2.0	α = 1.0	电子云范围扩大
权重归一化	直接使用密度值	归一化到 0.5-1.5	小原子也可见
密度归一化	线性归一化	对数归一化	动态范围压缩
低密度阈值	0.1	0.02	更多原子可见
高密度阈值	0.8	0.5	内层更清晰

前端实现 (JavaScript + Three.js)

1. 主页面结构 (`frontend/index.html`)

```html
<div class="container">
    <!-- 左侧控制面板 -->
    <div class="control-panel">
        <div class="section">文件加载</div>
        <div class="section">显示控制</div>
        <div class="section">分子构建器</div>
        <div class="section">操作按钮</div>
        <div class="section">信息显示</div>
    </div>

    <!-- 右侧 3D 画布 -->
    <div class="canvas-container">
        <canvas id="canvas3d"></canvas>
        <div class="loading">正在加载...</div>
        <div class="hint">操作提示</div>
    </div>
</div>
```

2. 3D 可视化 (`frontend/visualizer.js`)

A. Three.js 场景初始化

```html
function initScene() {
    // 1. 创建场景
    scene = new THREE.Scene();
    scene.background = new THREE.Color(0x1a1a2e);

    // 2. 创建相机
    camera = new THREE.PerspectiveCamera(
        75,                                     // 视角
        window.innerWidth / window.innerHeight, // 宽高比
        0.1,                                    // 近裁剪面
        1000                                    // 远裁剪面
    );
    camera.position.set(0, 0, 20);

    // 3. 创建渲染器
    renderer = new THREE.WebGLRenderer({
        canvas: document.getElementById('canvas3d'),
        antialias: true,
        preserveDrawingBuffer: true  // 用于截图
    });

    // 4. 添加光源
    const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
    const directionalLight = new THREE.DirectionalLight(0xffffff, 0.8);
    scene.add(ambientLight, directionalLight);

    // 5. 添加轨道控制器
    controls = new THREE.OrbitControls(camera, renderer.domElement);

    // 6. 创建分子组
    moleculeGroup = new THREE.Group();
    scene.add(moleculeGroup);

    // 7. 渲染循环
    function animate() {
        requestAnimationFrame(animate);
```

```
        controls.update();
        renderer.render(scene, camera);
    }
    animate();
}
```

B. 渲染原子

```
function renderAtoms(atoms) {
    atoms.forEach((atom, index) => {
        // 1. 创建球体几何
        const radius = ELEMENT_RADII[atom.element] || 0.5;
        const geometry = new THREE.SphereGeometry(radius, 32, 32);

        // 2. 创建材质
        const color = ELEMENT_COLORS[atom.element] || 0xcccccc;
        const material = new THREE.MeshStandardMaterial({
            color: color,
            metalness: 0.3,
            roughness: 0.7
        });

        // 3. 创建网格
        const mesh = new THREE.Mesh(geometry, material);
        mesh.position.set(
            atom.position[0],
            atom.position[1],
            atom.position[2]
        );

        // 4. 添加到场景
        mesh.userData = { type: 'atom', index: index, element: atom.element };
        moleculeGroup.add(mesh);
        atomMeshes.push(mesh);
    });
}
```

C. 渲染化学键

```
function renderBonds(atoms, bonds) {
    bonds.forEach(bond => {
        const atom1 = atoms[bond.atom1_idx];
        const atom2 = atoms[bond.atom2_idx];

        const pos1 = new THREE.Vector3(...atom1.position);
        const pos2 = new THREE.Vector3(...atom2.position);
        const midpoint = pos1.clone().add(pos2).multiplyScalar(0.5);

        // 根据键类型设置半径和颜色
        const radius = {
            'single': 0.1,
            'double': 0.15,
            'triple': 0.2
        }[bond.type] || 0.1;

        // 创建两段圆柱（双色显示）
        createBondCylinder(pos1, midpoint, radius, ELEMENT_COLORS[atom1.element]);
```

```
        createBondCylinder(midpoint, pos2, radius, ELEMENT_COLORS[atom2.element]);
    });
}
```

D. 渲染电子云等值面

```
function renderIsosurfaces(isosurfaces) {
    // 低密度等值面（绿色半透明）
    if (isosurfaces.low_density && isosurfaces.low_density.verts.length > 0) {
        const mesh = createIsosurfaceMesh(
            isosurfaces.low_density.verts,
            isosurfaces.low_density.faces,
            0x00ff00,  // 绿色
            0.2        // 透明度
        );
        moleculeGroup.add(mesh);
        isosurfaceMeshes.low_density = mesh;
    }

    // 高密度等值面（紫色半透明）
    if (isosurfaces.high_density && isosurfaces.high_density.verts.length > 0) {
        const mesh = createIsosurfaceMesh(
            isosurfaces.high_density.verts,
            isosurfaces.high_density.faces,
            0x8000ff,  // 紫色
            0.3        // 透明度
        );
        moleculeGroup.add(mesh);
        isosurfaceMeshes.high_density = mesh;
    }
}

function createIsosurfaceMesh(vertices, faces, color, opacity) {
    const geometry = new THREE.BufferGeometry();

    // 转换顶点数据
    const positions = [];
    vertices.forEach(v => positions.push(v[0], v[1], v[2]));
    geometry.setAttribute('position',
        new THREE.Float32BufferAttribute(positions, 3));

    // 转换面索引
    const indices = [];
    faces.forEach(f => indices.push(f[0], f[1], f[2]));
    geometry.setIndex(indices);

    // 计算法线（用于光照）
    geometry.computeVertexNormals();

    // 创建材质
    const material = new THREE.MeshStandardMaterial({
        color: color,
        transparent: true,
        opacity: opacity,
        side: THREE.DoubleSide
    });

    return new THREE.Mesh(geometry, material);
```

```
}
```

3. UI 交互 (`frontend/ui.js`)

A. 多语言支持

```
const languages = {
    zh: {
        title: '🧬 分子可视化工具',
        fileLoadBtnText: '选择分子文件',
        moleculeInfo: '原子数: {atoms}, 化学键数: {bonds}',
        // ... 更多文本
    },
    en: {
        title: '🧬 Molecular Visualizer',
        fileLoadBtnText: 'Choose Molecule File',
        moleculeInfo: 'Atoms: {atoms}, Bonds: {bonds}',
        // ... 更多文本
    }
};

function setLanguage(lang) {
    currentLanguage = lang;
    // 更新所有文本元素
    document.getElementById('title').textContent = languages[lang].title;
    // ...
}

function getText(key, params = {}) {
    let text = languages[currentLanguage][key];
    // 替换参数 {atoms} -> 实际值
    Object.keys(params).forEach(param => {
        text = text.replace(`{${param}}`, params[param]);
    });
    return text;
}
```

B. 文件上传处理

```
async function loadFile(file) {
    // 1. 创建 FormData
    const formData = new FormData();
    formData.append('file', file);

    // 2. 显示加载动画
    showLoading(true);
    showInfo(getText('loadingText'), 'info');

    try {
        // 3. 发送 POST 请求
        const response = await fetch('http://localhost:5000/api/parse', {
            method: 'POST',
            body: formData
        });

        // 4. 解析响应
```

```
        const data = await response.json();

        if (data.error) {
            throw new Error(data.error);
        }

        // 5. 渲染分子
        renderMolecule(data);

        // 6. 显示成功信息
        showInfo(getText('moleculeInfo', {
            atoms: data.atoms.length,
            bonds: data.bonds.length
        }), 'success');

    } catch (error) {
        showInfo(getText('errorLoad') + ': ' + error.message, 'error');
    } finally {
        showLoading(false);
    }
}
```

4. 分子构建器 (`frontend/builder.js`)

```
// 构建模式状态
let builderMode = false;
let selectedAtomType = 'C';
let selectedAtoms = [];
let builtMolecule = { atoms: [], bonds: [] };

// 添加原子
function addAtom(element, position) {
    const atom = {
        element: element,
        position: [position.x, position.y, position.z]
    };

    builtMolecule.atoms.push(atom);

    // 创建 3D 网格
    const geometry = new THREE.SphereGeometry(ELEMENT_RADII[element], 32, 32);
    const material = new THREE.MeshStandardMaterial({
        color: ELEMENT_COLORS[element]
    });
    const mesh = new THREE.Mesh(geometry, material);
    mesh.position.copy(position);

    moleculeGroup.add(mesh);
    atomMeshes.push(mesh);
}

// 创建化学键
function createBond(atom1Index, atom2Index) {
    // 验证化合价
    if (!validateBond(atom1Index, atom2Index)) {
        showInfo(getText('invalidBond'), 'error');
        return;
    }
```

```
    builtMolecule.bonds.push({
        atom1_idx: atom1Index,
        atom2_idx: atom2Index,
        type: 'single'
    });

    // 渲染化学键
    renderBuiltMolecule();
}

// 化合价验证
function validateMolecule() {
    const VALENCY_LIMITS = { H: 1, O: 2, N: 3, C: 4 };

    builtMolecule.atoms.forEach((atom, idx) => {
        // 统计该原子的键数
        const bondCount = builtMolecule.bonds.filter(b =>
            b.atom1_idx === idx || b.atom2_idx === idx
        ).length;

        const limit = VALENCY_LIMITS[atom.element];

        if (bondCount > limit) {
            // 标记错误
            atomMeshes[idx].material.color.set(0xff0000);
            errors.push(getText('valencyError', {
                element: atom.element,
                max: limit
            }));
        }
    });

    return errors.length === 0;
}
```

🎯 使用说明 / Usage Guide

加载分子

1. **上传文件**:
   - 点击"选择分子文件"按钮
   - 选择 `.log` , `.mol` , 或 `.cif` 文件
   - 等待解析完成
2. **显示控制**:
   - ✅ 显示低密度电子云（绿色）
   - ✅ 显示高密度电子云（紫色）
   - ✅ 显示原子（彩色球体）
   - ✅ 显示化学键（圆柱）
3. **交互操作**:
   - 左键拖拽：旋转分子
   - 滚轮：缩放视图
   - 右键拖拽：平移视图

- 按 S 键：保存截图

分子构建器

1. 启用构建模式
2. 选择原子类型（C, H, O, N）
3. 点击 3D 视图添加原子
4. 选择两个原子
5. 点击"创建化学键"
6. 系统自动验证化合价

📝 算法原理 / Algorithm Principles

化学键识别算法

原理：基于原子间距离判断

```
if triple_min ≤ distance ≤ triple_max:
    return "triple"
elif double_min ≤ distance ≤ double_max:
    return "double"
elif single_min ≤ distance ≤ single_max:
    return "single"
else:
    return None
```

电子云生成算法

步骤：

1. **空间离散化**：创建 50×50×50 的 3D 网格
2. **密度计算**：对每个网格点，计算所有原子的贡献

   $$\rho(x,y,z) = \Sigma_i\ w_i \times \exp(-\alpha \times r_i^2)$$

3. **权重归一化**：将权重映射到 0.5-1.5 范围
4. **对数归一化**：压缩动态范围

   $$\rho' = \log(1 + \rho) / \max(\log(1 + \rho))$$

5. **等值面提取**：使用 Marching Cubes 算法
   - 低密度：阈值 0.02
   - 高密度：阈值 0.5

🐛 故障排除 / Troubleshooting

常见问题

1. **端口被占用**

```
# Windows
netstat -ano | findstr :5000
taskkill /PID <PID> /F

# Linux/macOS
lsof -i :5000
kill -9 <PID>
```

2. **依赖安装失败**

```
# 使用镜像源
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```